



MemberDAO Protocol

Relatório Técnico — Desenvolvimento de Protocolo Web3 Completo

Unidade 1 · Capítulo 5 · Residência em TIC 29 — Web 3.0

Prof. Bruno Portes

Maio de 2025

Sumário

- 1 Introdução e Problema
- 2 Arquitetura do Protocolo
- 3 Implementação Técnica
 - 3.1 GovToken — ERC-20
 - 3.2 MemberPassNFT — ERC-721
 - 3.3 PriceConsumer — Oráculo Chainlink
 - 3.4 StakingPool
 - 3.5 MemberDAO — Governança
- 4 Segurança
- 5 Integração com Oráculo
- 6 Frontend Web3
- 7 Deploy na Sepolia
- 8 Auditoria
- 9 Conclusão

1. Introdução e Problema

O **MemberDAO Protocol** é um MVP (Minimum Viable Product) de protocolo descentralizado desenvolvido como entrega da Fase 2 Avançada da Residência em TIC 29 — Web 3.0. O protocolo integra os principais padrões e tecnologias do ecossistema Ethereum em um sistema coeso e funcional, deployado na testnet Sepolia.

O protocolo resolve três problemas recorrentes em comunidades descentralizadas:

> Falta de incentivo à participação

Membros passivos não contribuem com a comunidade. O mecanismo de staking recompensa quem mantém GTK em stake com novos tokens, criando um ciclo virtuoso de engajamento.

> Governança sem barreiras de qualidade

Qualquer endereço pode votar em decisões críticas sem comprometimento real. O NFT MemberPass atua como credencial de acesso, garantindo que apenas membros ativos participem do protocolo.

> Recompensas descoladas do mercado

Sistemas de staking com APY fixo tornam-se inflacionários em períodos de alta. A integração com o oráculo Chainlink ajusta dinamicamente a taxa de recompensa com base no preço de ETH/USD, tornando o protocolo financeiramente sustentável.

2. Arquitetura do Protocolo

O protocolo é composto por cinco contratos inteligentes organizados em duas camadas: contratos independentes (sem dependências entre si) e contratos dependentes (que consomem os primeiros). A camada de apresentação é um frontend HTML/JS que se conecta via ethers.js v6 e MetaMask.

Contrato	Padrão	Camada	Responsabilidade
GovToken	ERC-20 ERC20Votes	Independente	Token de governança e recompensa
MemberPassNFT	ERC-721	Independente	Credencial de acesso ao protocolo
PriceConsumer	Chainlink V3	Independente	Feed de preço ETH/USD on-chain
StakingPool	—	Dependente	Pool de staking com recompensa dinâmica
MemberDAO	—	Dependente	Governança: propostas e votação

Tabela 1 — Contratos do protocolo e suas responsabilidades.

Fluxo principal do usuário

- Mintar o **MemberPassNFT** (0,01 ETH) — obtém credencial de membro

- Adquirir **GTK** via transferência ou compra
- Aprovar o **StakingPool** a gastar GTK e realizar o stake
- Acumular recompensas ajustadas pelo preço ETH/USD (Chainlink)
- Sacar stake e recompensas via **exit()**
- Usar GTK para criar propostas e votar na **MemberDAO**

3. Implementação Técnica

Todos os contratos foram escritos em Solidity **^0.8.20** utilizando as bibliotecas da OpenZeppelin v5 e Chainlink Contracts v1.2. O ambiente de desenvolvimento utilizado é o **Hardhat** com o plugin *hardhat-toolbox*, com otimizador ativado (200 runs) e *evmVersion* configurado para **paris** para compatibilidade com a Sepolia.

3.1 GovToken (GTK) — ERC-20

O **GovToken** é o token nativo do protocolo. Herda de cinco contratos OpenZeppelin: *ERC20*, *ERC20Burnable*, *ERC20Votes*, *ERC20Permit* e *AccessControl*.

A extensão **ERC20Votes** adiciona checkpoints de saldo a cada transferência, permitindo que a MemberDAO consulte o poder de voto de cada endereço de forma historicamente precisa (resistente a flash loans). O **MINTER_ROLE** é concedido exclusivamente ao StakingPool no momento do deploy, garantindo que novos tokens só possam ser criados como recompensa legítima de staking.

Supply inicial: **1.000.000 GTK** mintados ao deployer para bootstrap de liquidez.

3.2 MemberPassNFT (MPASS) — ERC-721

O **MemberPassNFT** é a credencial de acesso ao protocolo. Cada endereço pode mintar exatamente **1 NFT**, controlado pelo mapping *hasMinted*. O preço de mint é de 0,01 ETH, configurável pelo owner.

O contrato implementa um mecanismo opcional de **soulbound**: quando ativado, o NFT torna-se intransferível após o mint, reforçando a semântica de identidade de membro único. O StakingPool verifica *memberPass.balanceOf(msg.sender) > 0* antes de aceitar qualquer depósito.

3.3 PriceConsumer — Oráculo Chainlink

O **PriceConsumer** consome o feed **ETH/USD** da Chainlink na Sepolia (endereço: 0x694AA1769357215DE4FAC081bf1f309aDC325306). O contrato implementa quatro validações de segurança sobre o dado retornado pelo feed antes de expô-lo ao StakingPool:

- Preço maior que zero
- Round ID não nulo (round completo)
- *answeredInRound* $\geq$ *roundId* (sem round desatualizado)
- Timestamp de atualização dentro de 3600 segundos (anti-staleness)

Em caso de falha do feed, o StakingPool captura a exceção via *try/catch* e aplica o **BASE_RATE** como fallback, garantindo continuidade do protocolo.

3.4 StakingPool

O StakingPool é o núcleo econômico do protocolo. Implementa o modelo **reward-per-token acumulado**, o mesmo padrão utilizado pelo protocolo Synthetix, que permite calcular recompensas individuais em O(1) independentemente do número de stakers.

Fórmula de recompensa

O acumulador global *rewardPerTokenStored* cresce continuamente com o tempo:

```
rewardPerToken = rewardPerTokenStored + (rate × Δtempo × 1e18) / totalStaked
```

A recompensa individual de cada usuário é calculada como:

```
earned = pendingRewards + staked × (rewardPerToken - rewardPerTokenPaid) / 1e18
```

Ajuste dinâmico pelo oráculo

A taxa de recompensa é inversamente proporcional ao preço de ETH, com limites mínimo e máximo:

```
rate = BASE_RATE × 1e8 / ethPrice  
rate = clamp(rate, MIN_RATE, MAX_RATE)
```

Quando ETH está caro, a emissão de GTK é reduzida (desinflacionário). Quando ETH está barato, a emissão aumenta para incentivar participação.

Parâmetro	Valor	Descrição
BASE_RATE	1e18 GTK/s	Taxa base quando ETH = \$1
MIN_RATE	1e16 GTK/s	Mínimo absoluto (0,01 GTK/s)
MAX_RATE	5e18 GTK/s	Máximo absoluto (5 GTK/s)

Tabela 2 — Parâmetros de recompensa do StakingPool.

Funções principais

`stake(uint256)`

Deposita GTK. Verifica NFT, atualiza acumulador, transfere tokens.

`withdraw(uint256)`

Retira GTK parcialmente. Recompensas permanecem pendentes.

`claimRewards()`

Minta GTK acumulados. Zera pendentes antes do mint (CEI).

`exit()`

Retira stake completo + recompensas em uma transação.

`emergencyWithdraw()`

Retira stake sem coletar recompensas (sem acumulador).

3.5 MemberDAO — Governança

A **MemberDAO** implementa um mecanismo de governança simplificado. Qualquer holder de GTK com saldo mínimo de **1.000 GTK** pode criar propostas. O peso do voto é proporcional ao saldo GTK no momento da votação.

O ciclo de vida de uma proposta percorre os estados: **Active** → **Passed/Rejected** → **Executed/Cancelled**. Uma proposta é aprovada se obtiver maioria simples de votos favoráveis e atingir o quórum mínimo de **100.000 GTK**. O período de votação padrão é de **3 dias**, todos os parâmetros são configuráveis pelo owner via *updateGovernanceParams()*.

4. Segurança

A segurança do protocolo foi tratada como requisito de primeira classe, com múltiplas camadas de proteção aplicadas em todos os contratos.

✓ Padrão CEI (Checks-Effects-Interactions)

Todas as funções que movem valor seguem estritamente a ordem: verificações com `require`, atualização de estado, e só então interações externas (`transfer`, `mint`). Isso elimina a janela de ataque de reentrância na camada lógica.

✓ ReentrancyGuard (OpenZeppelin)

Adicionado como segunda camada de proteção em todas as funções públicas do `StakingPool` que movem tokens (`stake`, `withdraw`, `claimRewards`, `exit`, `emergencyWithdraw`). O modificador `nonReentrant` bloqueia chamadas reentrantes via mutex de estado.

✓ AccessControl (MINTER_ROLE)

Somente o endereço com `MINTER_ROLE` pode chamar `govToken.mint()`. Esse papel é concedido exclusivamente ao `StakingPool` no deploy, impedindo inflação arbitrária do supply de GTK.

✓ Validação do oráculo (anti-staleness)

O `PriceConsumer` rejeita dados com mais de 3600 segundos, dados de rounds incompletos e preços não positivos. O `StakingPool` usa `try/catch` para não travar em caso de falha do feed.

✓ Solidity ^0.8.20

A partir da versão 0.8.0, overflow e underflow em aritmética inteira causam `revert` automaticamente, eliminando a necessidade de `SafeMath` e prevenindo uma classe inteira de vulnerabilidades.

✓ Soulbound opcional no NFT

O `MemberPassNFT` implementa bloqueio de transferência configurável, impedindo a venda de credenciais de membro no mercado secundário quando ativado.

5. Integração com Oráculo

A integração com o oráculo **Chainlink Price Feeds** é feita através da interface *AggregatorV3Interface*, consumindo o par **ETH/USD** com 8 casas decimais.

O feed utilizado na Sepolia é o endereço oficial **0x694AA1769357215DE4FAC081bf1f309aDC325306**, mantido pela Chainlink Labs. A atualização do preço ocorre automaticamente a cada interação do usuário com o `StakingPool`, sem necessidade de chamadas externas ou keepers.

A relação entre preço e taxa de recompensa é **inversamente proporcional**: quando o preço de ETH sobe, menos GTK são emitidos por segundo, reduzindo a pressão inflacionária sobre o token. Isso cria um equilíbrio econômico natural onde o protocolo se torna mais conservador exatamente quando o mercado está aquecido.

6. Frontend Web3

O frontend é uma single-page application em HTML/CSS/JavaScript puro, sem frameworks ou bundlers, para máxima portabilidade. A conexão com a blockchain é feita via **ethers.js v6** através do provedor injetado pelo MetaMask (*window.ethereum*).

Funcionalidade	Descrição
Aba NFT	Mint do MemberPass com verificação de posse e exibição de status
Aba Token	Consulta de saldo, transferência de GTK e delegação de votos
Aba Staking	Stake, withdraw, claimRewards e exit com aprovação automática
Aba DAO	Criação de propostas, votação ponderada e finalização de propostas expiradas
Aba Info	Configuração e persistência dos endereços dos contratos via localStorage
Stats em tempo real	Painel com saldo GTK, stake, recompensas e preço ETH — atualizado a cada 15s
Validação de rede	Deteção automática de rede incorreta com botão de troca para Sepolia
Verificação on-chain	getCode() valida que cada endereço contém um contrato antes de chamar

Tabela 3 — Funcionalidades do frontend.

7. Deploy na Sepolia Testnet

O deploy é realizado via script Hardhat (*scripts/deploy.js*) que orquestra o deployment sequencial dos cinco contratos e realiza a configuração de permissões automaticamente. Ao final, gera o arquivo *frontend/addresses.json* com todos os endereços e links do Etherscan.

Etapa	Ação	Detalhe
1	Deploy GovToken	Minta 1.000.000 GTK ao deployer
2	Deploy MemberPassNFT	Define URI base dos metadados
3	Deploy PriceConsumer	Conecta ao feed Chainlink ETH/USD Sepolia
4	Deploy StakingPool	Recebe endereços dos 3 contratos anteriores
5	Deploy MemberDAO	Recebe endereço do GovToken

6	Configuração	grantRole(MINTER_ROLE, StakingPool)
7	Exportação	Salva addresses.json com links do Etherscan

Tabela 4 — Sequência de deploy na Sepolia.

Configuração do Hardhat

O arquivo *hardhat.config.js* foi configurado com:

- Solidity 0.8.20 com otimizador (200 runs)
- evmVersion: 'paris' — necessário para compatibilidade com OpenZeppelin v5 na Sepolia
- Rede Sepolia com RPC via Infura ou Alchemy (variável `SEPOLIA_RPC`)
- Integração com Etherscan para verificação automática de contratos

8. Auditoria de Segurança

A auditoria foi realizada utilizando **Mythril v0.24.8**, ferramenta de análise de segurança baseada em execução simbólica que explora matematicamente todos os caminhos de execução possíveis de um contrato.

Metodologia

Devido à natureza da execução simbólica, a análise de cada contrato pode levar entre 2 e 40 minutos dependendo da complexidade. O workflow adotado foi:

- Compilação via Hardhat: `npx hardhat compile`
- Extração do bytecode deployado de cada artefato JSON
- Análise com timeout de 120s e profundidade máxima de 10 (equilíbrio velocidade/cobertura)
- Análise via endereço on-chain usando RPC da Sepolia para os contratos já deployados

Vulnerabilidades verificadas (SWC)

SWC ID	Vulnerabilidade	Contrato	Mitigação
SWC-107	Reentrância	StakingPool	ReentrancyGuard + CEI
SWC-101	Integer overflow	StakingPool	Solidity ^0.8.20 (built-in)
SWC-104	Unchecked return	Todos	require(ok, ...) em transfers
SWC-115	tx.origin auth	Todos	msg.sender em todos os checks
SWC-116	Timestamp dep.	MemberDAO	Janela de 3 dias — aceitável
SWC-124	Write arb. storage	Todos	AccessControl + Ownable

Tabela 5 — Matriz de vulnerabilidades e mitigações.

Falsos positivos esperados

A execução simbólica pode gerar alertas sobre o uso de `block.timestamp` no contrato MemberDAO (deadline das propostas). Esta dependência é considerada **aceitável**: um minerador pode manipular o timestamp em aproximadamente 15 segundos, o que é insignificante frente a um período de votação de 3 dias. Este alerta deve ser documentado como falso positivo no relatório de auditoria final.

9. Conclusão

O **MemberDAO Protocol** entrega um MVP funcional e coeso que integra todos os componentes exigidos pela Fase 2 Avançada da Residência em TIC 29:

Critério	Entrega	Peso
Arquitetura e Modelagem	Diagrama completo, justificativa de padrões ERC, fluxo documentado	20%
Implementação Técnica	5 contratos Solidity ^0.8.20 com OpenZeppelin v5	20%
Segurança	CEI, ReentrancyGuard, AccessControl, validação de oráculo	20%
Integração Oráculo	Chainlink ETH/USD com anti-staleness e fallback	10%
Integração Web3	Frontend ethers.js v6 + deploy script Hardhat	10%
Deploy em Testnet	Deploy na Sepolia com endereços verificados no Etherscan	10%
Clareza do Relatório	Relatório técnico em PDF com todas as seções	10%

Tabela 6 — Rubrica de avaliação e entregáveis correspondentes.

O protocolo demonstra como os primitivos do ecossistema Ethereum — tokens fungíveis, NFTs, oráculos e governança — podem ser compostos de forma segura e economicamente coerente em um sistema descentralizado completo. A arquitetura em camadas, com contratos independentes servindo de base para os dependentes, facilita a auditoria, o teste unitário e futuras extensões do protocolo.